

Azure Service Bus – Message Operations

Azure Service Bus provides different operations to handle messages after a receiver receives them.

The four important message operations are:

1. **Complete**
2. **Abandon**
3. **Defer**
4. **Dead-Letter**

1. Complete Message

Meaning

When the receiver successfully processes a message, it marks the message as **Complete**.

Once the message is completed:

The message is permanently removed from the queue.

Example

Suppose the queue contains:

Message 1 → Process Payment

The receiver receives and successfully processes it.

```
Queue
  ↓
Receive Message
  ↓
Process Successfully
  ↓
Complete
  ↓
Message Deleted
```

When do we use Complete?

Use **Complete** when:

- The message was successfully processed.
- No further processing is required.
- The operation was successful.

Simple Example

```
await receiver.CompleteMessageAsync(message);
```

Remember

Complete = Successfully processed → Delete message

2. Abandon Message

Meaning

When a receiver **abandons** a message, the message remains in the queue.

It becomes available for processing again.

The **delivery count increases by 1**.

Example

Suppose a receiver receives a message but realizes that it cannot process it.

Instead of completing the message, it abandons it.

```
Queue
↓
Receive Message
↓
Cannot Process
↓
Abandon
↓
Message remains in Queue
```

↓
Available again

Another receiver can then pick up the message.

When do we use Abandon?

Use **Abandon** when:

- The message cannot be processed right now.
- The receiver does not need the message anymore.
- You want the message to become available for processing again.

Example Code

```
await receiver.AbandonMessageAsync(message);
```

Important

When a message is abandoned:

```
Delivery Count = Delivery Count + 1
```

Remember

Abandon = Don't process now → Keep message in queue → Available again

3. Defer Message

Meaning

Defer means:

Keep the message aside temporarily and process it later.

A deferred message is not normally available through the regular receive operation.

The receiver can retrieve the deferred message later using its **sequence number**.

Real-Time Example

Suppose a service has two tasks:

Task 1 → Complete first
Task 2 → Complete after Task 1

The service receives a message related to **Task 2**.

But Task 1 has not been completed yet.

Therefore, the service cannot process Task 2.

Instead of abandoning the message, it can **defer** the message.

```
Task 1
  ↓
Not completed
  ↓
Task 2 message received
  ↓
Cannot process Task 2
  ↓
Defer message
  ↓
Task 1 completed
  ↓
Retrieve deferred Task 2 message
  ↓
Process Task 2
```

Why use Defer?

Use **Defer** when:

- The message is valid.
- You want to process it later.
- Another operation must happen first.
- The message needs to wait for some prerequisite.

Example Code

```
await receiver.DeferMessageAsync(message);
```

Later, the application can retrieve the deferred message using its sequence number.

```
var message =  
    await receiver.ReceiveDeferredMessageAsync(sequenceNumber);
```

Important

When a message is deferred:

- It remains in Service Bus.
- It is kept aside for later processing.
- The receiver can retrieve it later.
- The message's delivery count can increase when it is subsequently received.

Remember

Defer = Process later → Keep aside → Retrieve later

4. Dead-Letter Message

Meaning

A **dead-letter message** is a message that has been moved to the **Dead-Letter Queue (DLQ)** because it cannot or should not be processed normally.

The message is not immediately deleted.

It can still be retrieved from the Dead-Letter Queue.

Example

Suppose a message repeatedly fails processing.

```
Message  
↓  
Receive  
↓  
Processing fails  
↓  
Retry  
↓  
Processing fails  
↓  
Retry  
↓
```

Too many failures
↓
Dead-Letter Queue

The application can later inspect the message from the DLQ.

Why use Dead-Letter Queue?

The Dead-Letter Queue is commonly used for:

- Troubleshooting
- Debugging
- Investigating failed messages
- Handling messages that cannot be processed
- Manual review of problematic messages

Example Code

```
await receiver.DeadLetterMessageAsync(message);
```

The message is then available in the Dead-Letter Queue.

Remember

Dead-Letter = Problematic message → Move to DLQ → Investigate later

5. Difference Between Complete, Abandon, Defer and Dead-Letter

Operation	What happens to message?	When to use?
Complete	Message is deleted	Successfully processed
Abandon	Message remains available for processing	Cannot process now / retry
Defer	Message is kept aside for later retrieval	Need to process after another task
Dead-Letter	Message moves to DLQ	Message cannot/should not be normally processed

6. Easy Way to Remember

Think about a restaurant order.

Complete

You successfully finish the order.

Order → Completed → Removed

Complete = Done

Abandon

You cannot handle the order, so you put it back for someone else.

Order → Abandoned → Available again

Abandon = Try again

Defer

You need to wait for something before handling the order.

Order → Deferred → Process later

Defer = Wait

Dead-Letter

There is a serious problem with the order, so you move it to a special area for investigation.

Order → Dead Letter → Investigate

Dead-Letter = Problem / Investigation

7. Important Interview Scenario

Question:

What is the difference between Abandon and Defer?

Answer:

Abandon:

The message remains available in the queue and can be picked up again by another receiver. The delivery count increases.

Defer:

The message is intentionally kept aside because it cannot be processed yet. The application can retrieve it later using its sequence number.

Simple Difference

Abandon

↓

Message becomes available again

↓

Any eligible receiver can process it

Defer

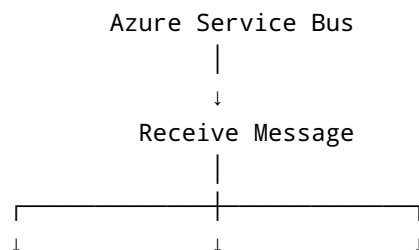
↓

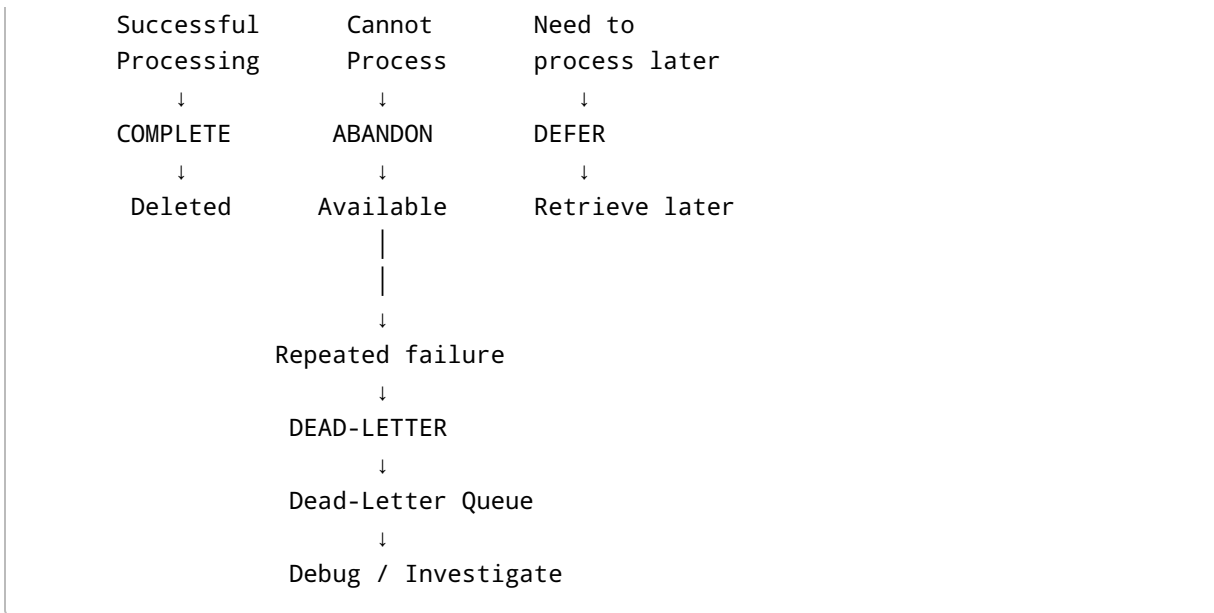
Message is kept aside

↓

Application retrieves it later

8. Complete Flow





9. Key Points for Interview

Complete

- Used after successful processing.
- Message is removed from the queue.

```
await receiver.CompleteMessageAsync(message);
```

Abandon

- Message remains available.
- Can be processed again.
- Delivery count increases.

```
await receiver.AbandonMessageAsync(message);
```

Defer

- Message is intentionally kept aside.
- Used when the message cannot be processed yet.
- Retrieve it later using its sequence number.

```
await receiver.DeferMessageAsync(message);
```

Dead-Letter

- Moves the message to the Dead-Letter Queue.
- Useful for problematic messages.
- Useful for debugging and investigation.

```
await receiver.DeadLetterMessageAsync(message);
```

10. One-Minute Interview Answer

If the interviewer asks:

"What are the different message operations in Azure Service Bus?"

You can answer:

Azure Service Bus provides operations such as Complete, Abandon, Defer, and Dead-Letter. Complete is used when the message is successfully processed, and the message is removed from the queue. Abandon is used when the message cannot be processed and should become available again. Defer is used when the message is valid but needs to be processed later because some prerequisite operation has not completed. Dead-Letter is used for problematic messages that should be moved to the Dead-Letter Queue for troubleshooting or further investigation.

11. Quick Revision

COMPLETE

- Successfully processed
- Message deleted

ABANDON

- Cannot process now
- Message available again
- Delivery count increases

DEFER

- Need to process later
- Keep message aside
- Retrieve later using sequence number

DEAD-LETTER

→ Problematic message

→ Move to DLQ

→ Debug / Investigate

Easy Memory Trick

Complete = Done

Abandon = Try Again

Defer = Later

Dead-Letter = Problem / Investigate